

Hardening npm Publishing

サプライチェーン侵害から公開フローを守る

azu (@azu_re)

2026-06-23 / OSS開発者は今何をすべきか？

自己紹介

- Name : azu
- X/Twitter : @[azu_re](#)
- Website : [Web scratch](#), [JSer.info](#)
- Open Source : textlint / secretlint / HonKit など
- npmに公開しているパッケージは数百個



なぜ今、公開フローを守るのか

- npmは依存が深く、1つのパッケージ侵害が広い範囲に波及する
- 攻撃の起点がメンテナのトークンやCI/CDに移ってきている
- OIDCトークンの窃取、不正なworkflow、infostealerが実際に起きている
- 自分のパッケージが踏み台になる前提で守る

問い: 公開フローのどこを守るか

- 攻撃者は、1箇所のインジェクトで全部抜けるなら一番弱い部分を狙う
- だから1つの対策だけで守れる魔法の弾丸は存在しない
- ローカルから公開までの流れを1個ずつ塞ぐ
- どこかが壊れても、別のどこかで止める

公開までのレイヤー

1. ローカルのToken管理
2. トークンレスnpm (OIDC Trusted Publishing)
3. GitHub Actions (Rulesets / Environments / Deployment protection)
4. OIDCと権限昇格への対策
5. npm staged publishing

1. ローカルのToken管理

生のcredentialをローカルに置かない

- 第一前提は、infostealerにローカルを抜かれても公開権限が漏れないこと
- ローカルファイルにsecretを保存しない（昔のパスワード管理の延長）
- 強いトークンをローカルに常駐させない

GitHub: classic PATを常用しない

- classic PATは広い範囲にwriteできてしまい強すぎる
- 使うとしてもローカル限定。CIではほぼ使わない
- 常用はfine-grained PATにして、リポジトリと権限を絞る
- fine-grainedはリソースオーナーに紐づくので多少手間だが許容する

fine-grained PATは用途ごとに分ける

- 1つの強いトークンを使い回さず、用途ごとに発行する
- アクセスレベルを最小にする（read-only / 必要な操作だけ）
- 対象を絞る（特定リポジトリ / public のみ）
- 漏れても影響範囲がそのトークンの用途に限定される

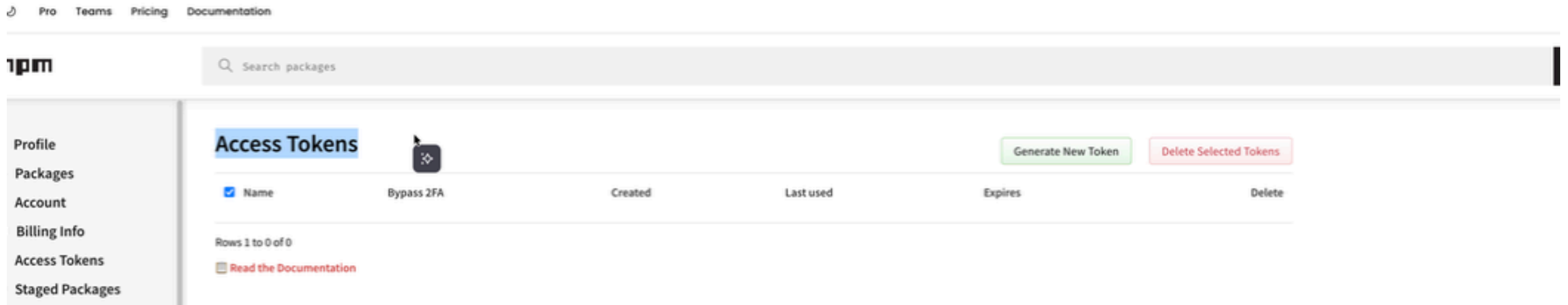
例: pull用read-only、deploy用、uptime監視用を別々に持つ

課題: Checks APIがfine-grained未対応

- Checks APIがfine-grained PATに対応しておらず、classicがfallbackになる
- これが直れば、常用トークンから強いclassicをほぼ消せる

参考: github.com/orgs/community/discussions/129512

npm: アクセストークンを0個にする



Require 2FA and disallow tokens

- npmのパッケージ設定で「2FA必須・トークン公開を禁止」にする
- 既存パッケージはトークンでは公開できなくなる
- 公開経路はOIDC（Actions経由）だけに強制される

2. トークンレス npm

OIDC Trusted Publishing

OIDC Trusted Publishingとは

- 長期トークンをやめ、short-livedでworkflow固有の署名トークンで公開する
- npmとGitHub ActionsがOIDCで信頼関係を結ぶ
- 「特定リポジトリの特定workflowからの実行」を暗号的に証明する
- npm 11.5.1以上が必要

参考: efcl.info/2025/09/07/npm-oidc/

npmjs.comでTrusted Publisherを登録

The screenshot shows the 'textlint' package page on npmjs.com. The 'Trusted Publisher' section is active, displaying a form to configure a trusted publisher using GitHub Actions. The form includes fields for 'Publisher' (set to GitHub Actions), 'Organization or user' (textlint), 'Repository' (textlint), 'Workflow filename' (release.yml), and 'Environment name' (npm). There are checkboxes for 'Allowed actions' (Allow npm publish is checked). The right sidebar shows package statistics: 153,506 weekly downloads, version 15.7.1, MIT license, and last published 23 days ago. At the bottom, there are buttons for 'Analyze security with Socket' and 'Check bundle size'.

Pro Teams Pricing Documentation

npm Search packages Search

textlint 15.7.1 • Public • Published 23 days ago

Readme Code (Beta) 24 Dependencies 38 Dependents 268 Versions Settings

Trusted Publisher

Establish a trust between your package and your repository using OpenID Connect (OIDC).
[Learn more about OpenID Connect.](#)

Publisher*

GitHub Actions

Organization or user* / **Repository***

textlint / textlint

Workflow filename*

Filename only (e.g., publish.yml). Must exist in ".github/workflows/" in your repository.

release.yml

Environment name

The name of the GitHub Actions environment used for publishing. This is encouraged for projects with maintainers who should not have npm publishing access.

npm

Allowed actions *

Select which actions this trusted publisher can perform. At least one must be selected.

☒ Allow npm publish

☐ Allow npm stage publish

Save changes Cancel

Package access

Status:

Install

> npm i textlint

Repository

github.com/textlint/textlint

Homepage

github.com/textlint/textlint/

Weekly Downloads

153,506

Version **License**

15.7.1 MIT

Last publish

23 days ago

Collaborators

Analyze security with Socket

Check bundle size

workflow側の最小構成

permissions:

contents: write

id-token: write # OIDC

steps:

- uses: actions/setup-node@v5

with:

registry-url: 'https://registry.npmjs.org'

- run: npm publish --access public

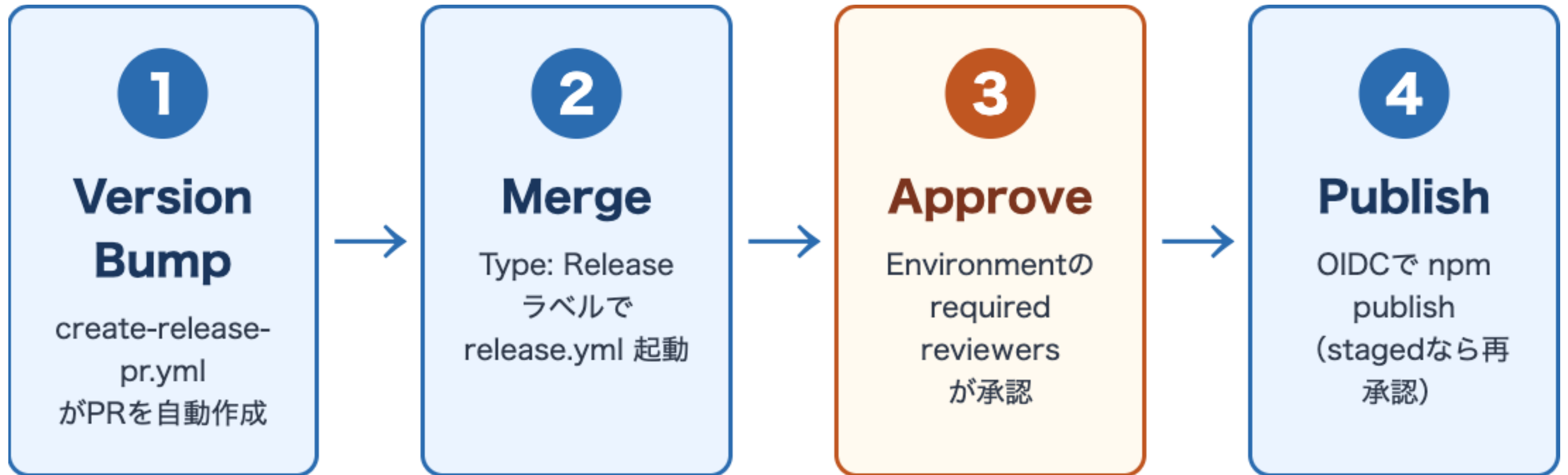
provenanceで来歴を残す

- OIDC公開ではprovenance（来歴の署名）が自動付与される
- どのリポジトリのどのworkflowでビルドされたかを証明できる
- 利用者はsigstoreで検証できる

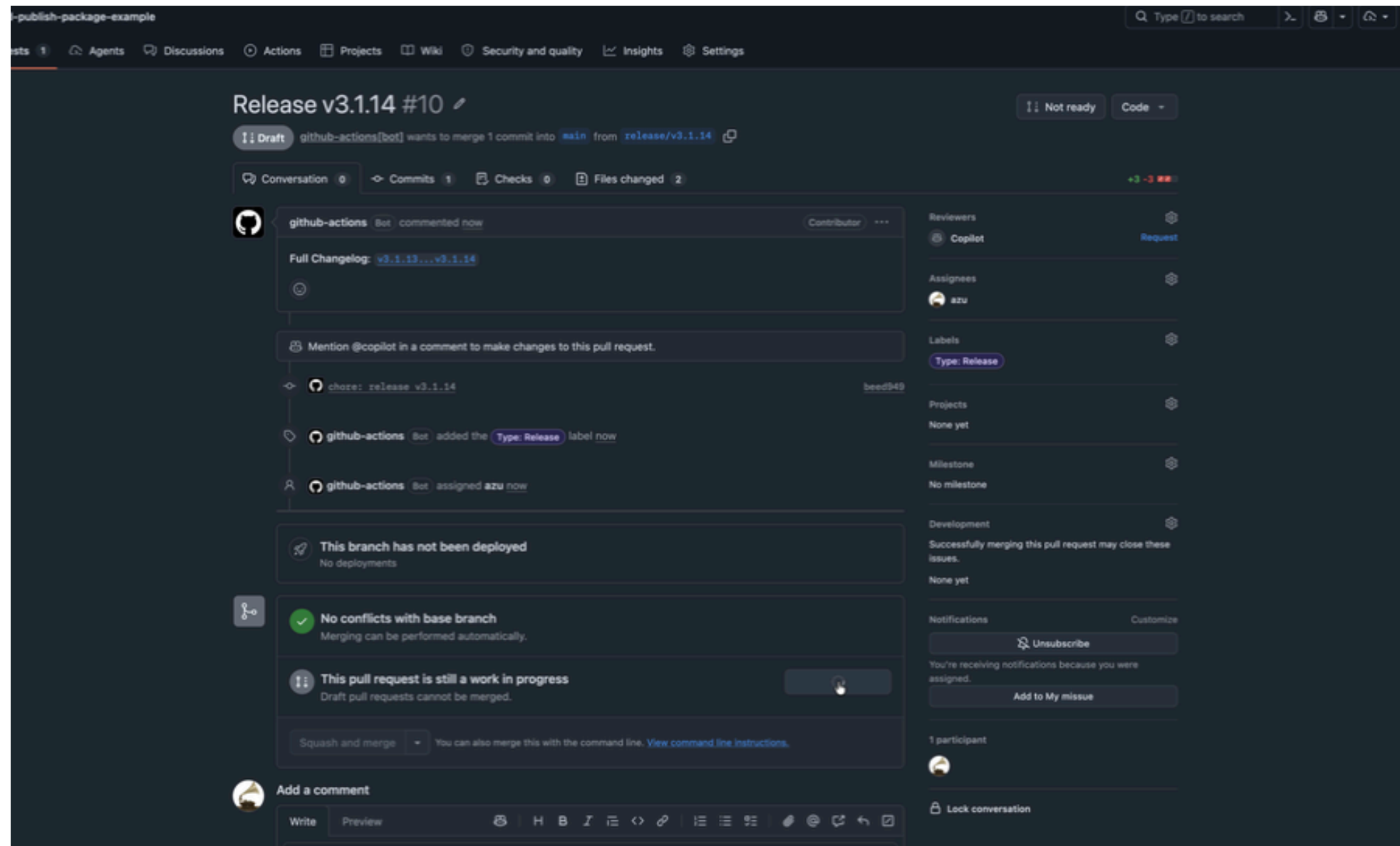
3. GitHub Actions の構成

Rulesets / Environments / Deployment
protection

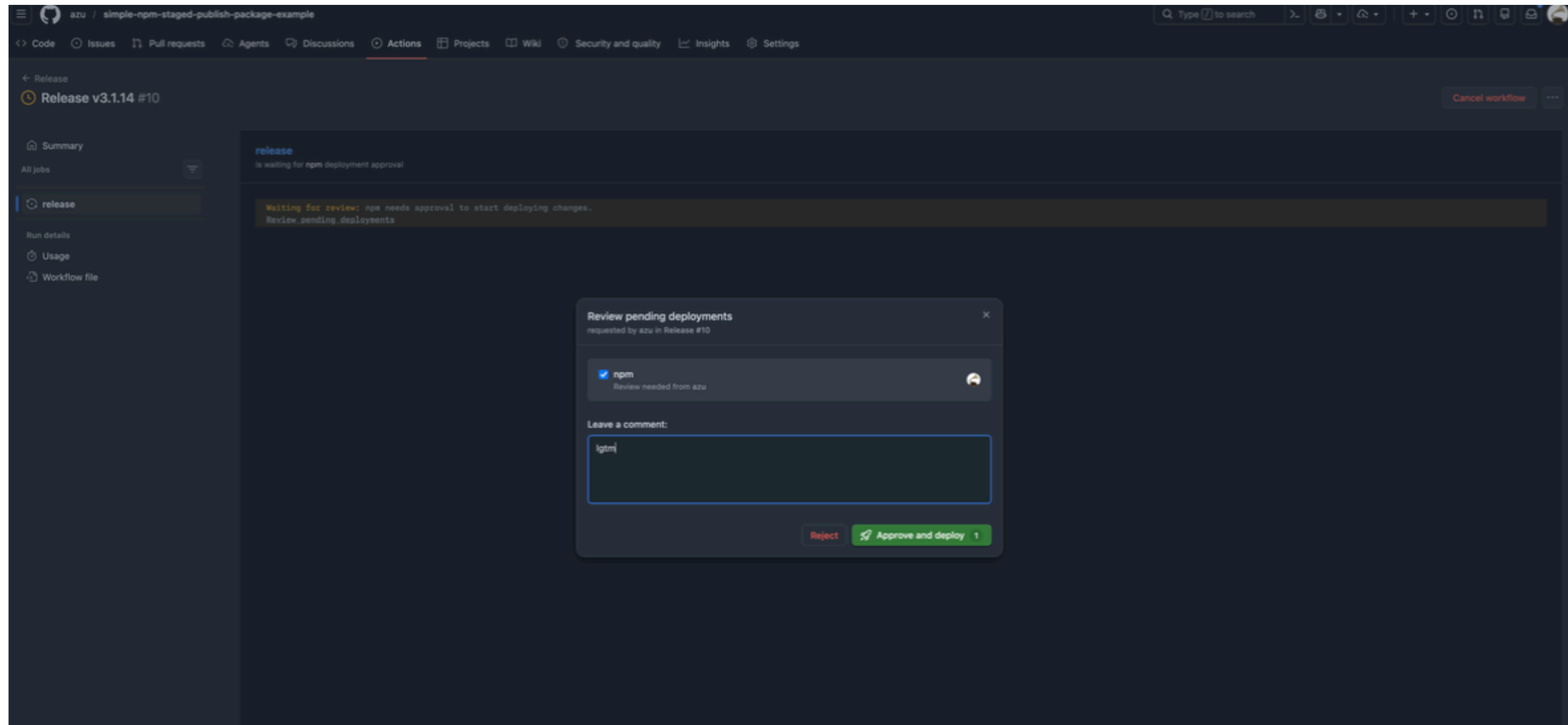
手順: Release PRの流れ



手順1: Version Up (バージョンアップPR)



手順3: Approve（承認）して公開する



実装: Release PR方式

- create release PRでリリース用のPRを作る
- マージ後に「Type: Release」ラベルが付いたときだけ release.yml が動く
- リリースは必ずPR経由を通す

参考: github.com/azu/simple-oidc-example-package

実装: release.yml (抜粋)

```
simple-oidc-example-package / .github / workflows / release.yml ↑ Top
Code Blame 128 lines (115 loc) · 4.21 KB · ⓘ
3   branches:
4     - master
5     - main
6   types:
7     - closed
8
9 jobs:
10
11   release:
12     if: |
13       github.event.pull_request.merged == true &&
14       contains(github.event.pull_request.labels.*.name, 'Type: Release')
15     runs-on: ubuntu-latest
16     environment:
17       name: npm
18     permissions:
19       contents: write
20       id-token: write # OIDC
21     outputs:
22       released: ${ steps.tag-check.outputs.exists == 'false' }
23       version: ${ steps.package.outputs.version }
24       package-name: ${ steps.package.outputs.name }
25       release-url: ${ steps.create-release.outputs.url }
26     steps:
27       - name: Checkout
28         uses: actions/checkout@08c6903cd8c0fde910a37f88322edcfb5dd907a8 # v5.0.0
29         with:
30           persist-credentials: false
31
32       - name: Get package info
33         id: package
34         run: |
35           VERSION=$(jq -r '.version' package.json)
36           PACKAGE_NAME=$(jq -r '.name' package.json)
37           echo "version=$VERSION" >> $GITHUB_OUTPUT
38           echo "name=$PACKAGE_NAME" >> $GITHUB_OUTPUT
39
40       - name: Check if tag exists
41         id: tag-check
42         run: |
43           if git rev-parse "v$VERSION" >/dev/null 2>&1; then
44             echo "exists=true" >> $GITHUB_OUTPUT
45           else
46             echo "exists=false" >> $GITHUB_OUTPUT
47           fi
48       env:
49         VERSION: ${ steps.package.outputs.version }
50
51       - name: Setup Node.js
52         if: steps.tag-check.outputs.exists == 'false'
53         uses: actions/setup-node@0853c24544627f65ddf259abe73b1d18a591444 # v5.0.0
54         with:
```

実装: 外部Actionに依存しない

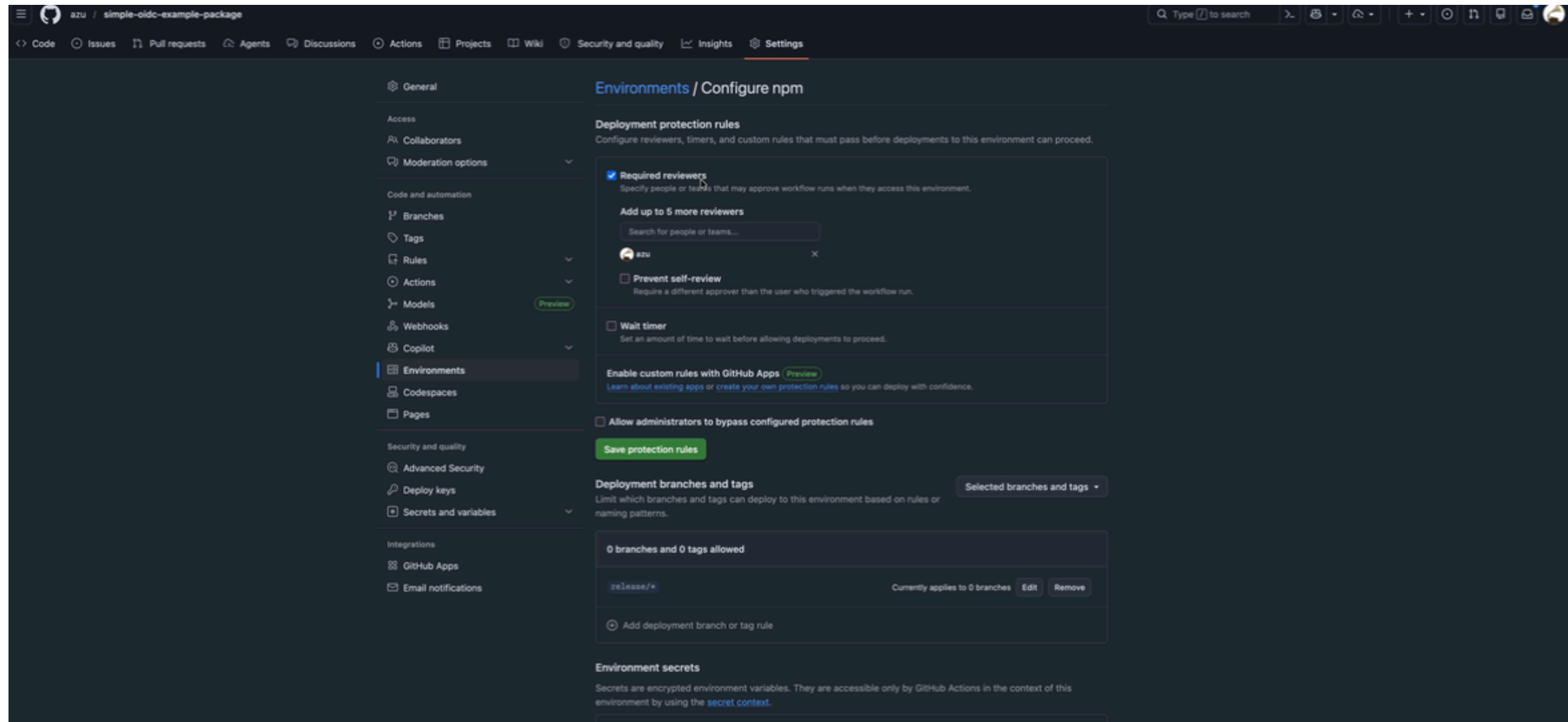
- 使うのは公式の checkout / setup-node のみ
- setup-nodeのcacheも使わない
- リポジトリ操作は gh / git コマンドで行う
- Actionはcommit SHAでpinし、persist-credentials: false にする

なぜcacheを使わないか (cache poisoning)

- GitHub Actionsのcacheは権限に関係なくどのworkflowからも読み書きできる
- 低権限やPRのworkflowがcacheを汚染し、リリースworkflowが復元して実行してしまう
- TanStack侵害: pullrequesttargetで汚染 → release.ymlで復元 → メモリからOIDCトークン窃取 → 84バージョン公開
- secretを扱うworkflowではcacheを消費しない

参考: [tanstack postmortem](#) / [clinejection](#)

実装: Environmentで承認ゲートを置く



実装: なぜ改変だけでは公開できないか

- npmのTrusted Publisherはworkflowファイル名とEnvironment名で縛る
- release.ymlを改変しても environment npm が使えない
- environmentにはrequired reviewersがあり承認が要る
- ファイル名一致だけではOIDCトークンを交換できない

4. OIDC と権限昇格

Bitwarden CLI侵害事例

- CI/CDが侵害され、メモリや環境変数からトークン・クラウド資格情報を窃取
- .github/workflows/ に不正workflowを作り、transient branchで検出回避
- 肝はEnvironmentを使っていなかったこと

参考: socket.dev/blog/bitwarden-cli-compromised

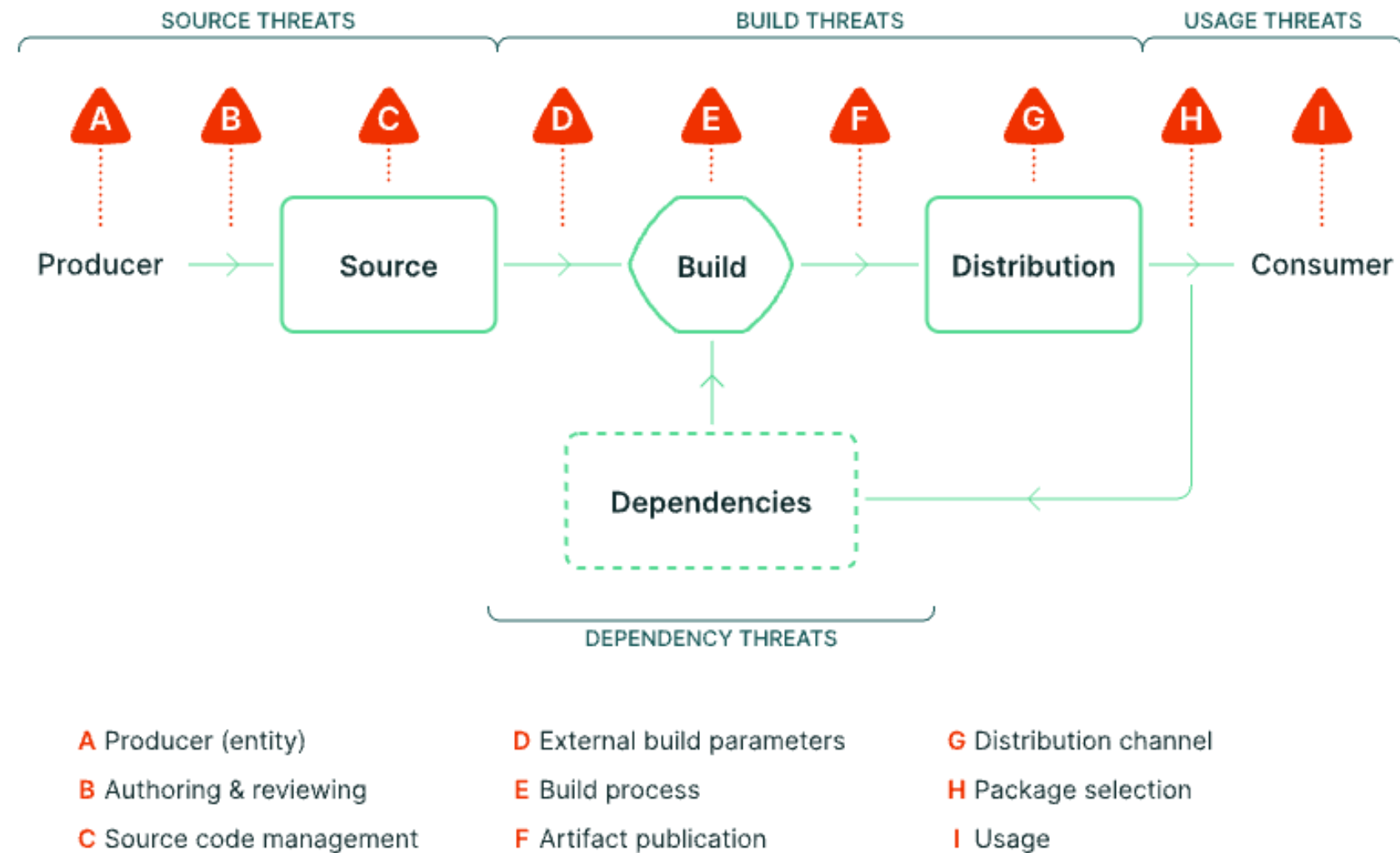
権限カスケード（権限昇格）

- workflow名一致だけでOIDC交換できる構成だと
- GitHubを侵害するだけでnpmも侵害できてしまう
- write権限しかないはずがnpm publish権限に化ける
- 対策は、権限が上がる箇所に承認フローを1個挟むこと

SLSAでレイヤーごとに対策する

- Source(A,B,C) / Build(D,E,F,G) / Usage(H,I)
- npm公開は主に (F) Artifact publication
- (F)は最も直接的で簡単な脅威とされる
- OIDC・environment・stagedはどれも (F)を多層で固める

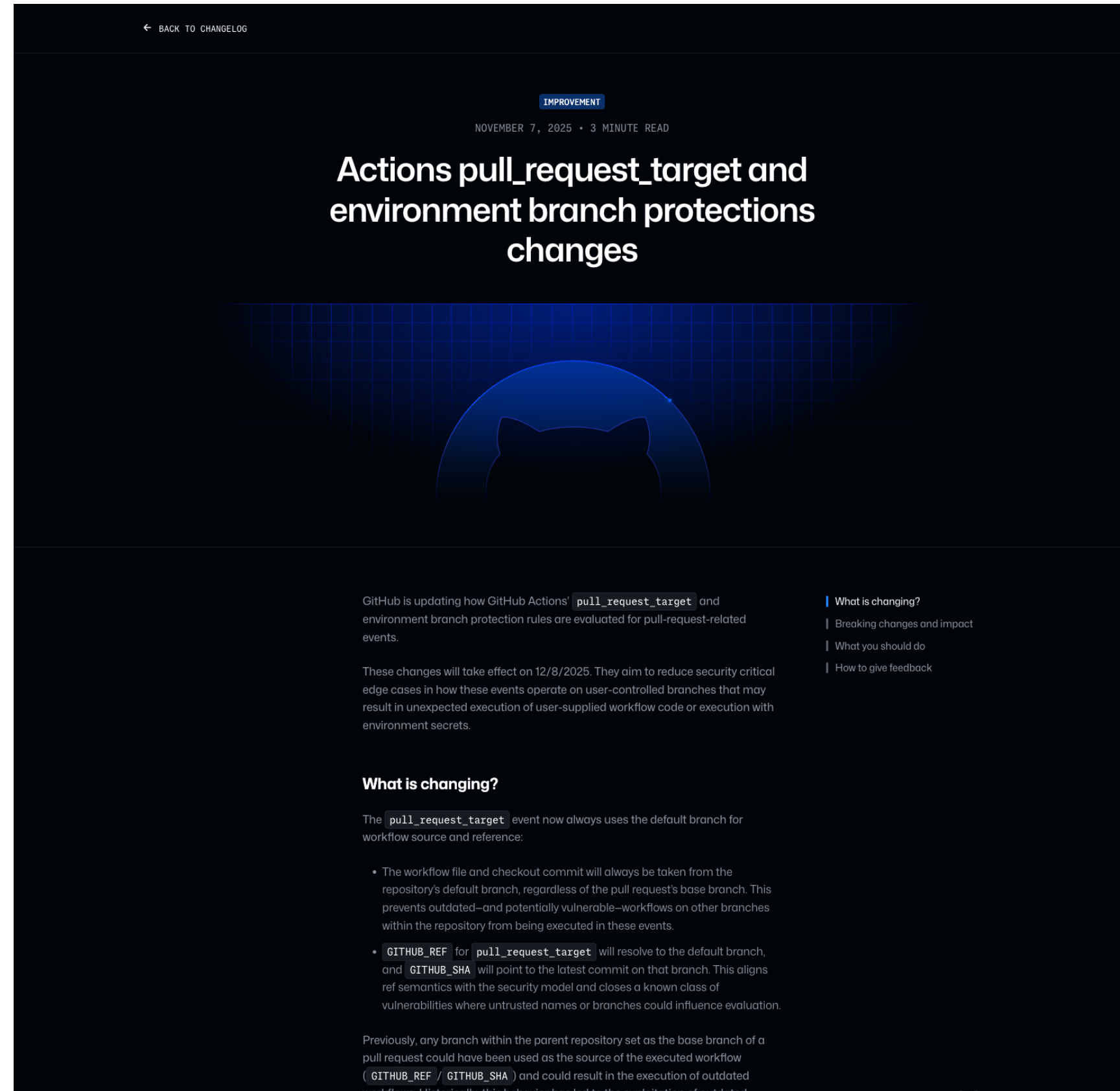
参考: slsa.dev/spec/v1.2/threats



ポイント: PR + マージを 必須にする

- 直接pushや単独のworkflow起動ではリリースさせない
- 必ずPRを作ってマージした文脈でだけ release.yml が動く
- environmentのブランチ保護も refs/pull/<n>/merge で評価される
- この辺りはGitHub側の仕様変更が続いている領域

参考: [github.blog changelog 2025-11-07](https://github.blog/changelog/2025-11-07-actions-pull-request-target-and-environment-branch-protections-changes/)



5. npm staged publishing

公開前に承認ステップを足す

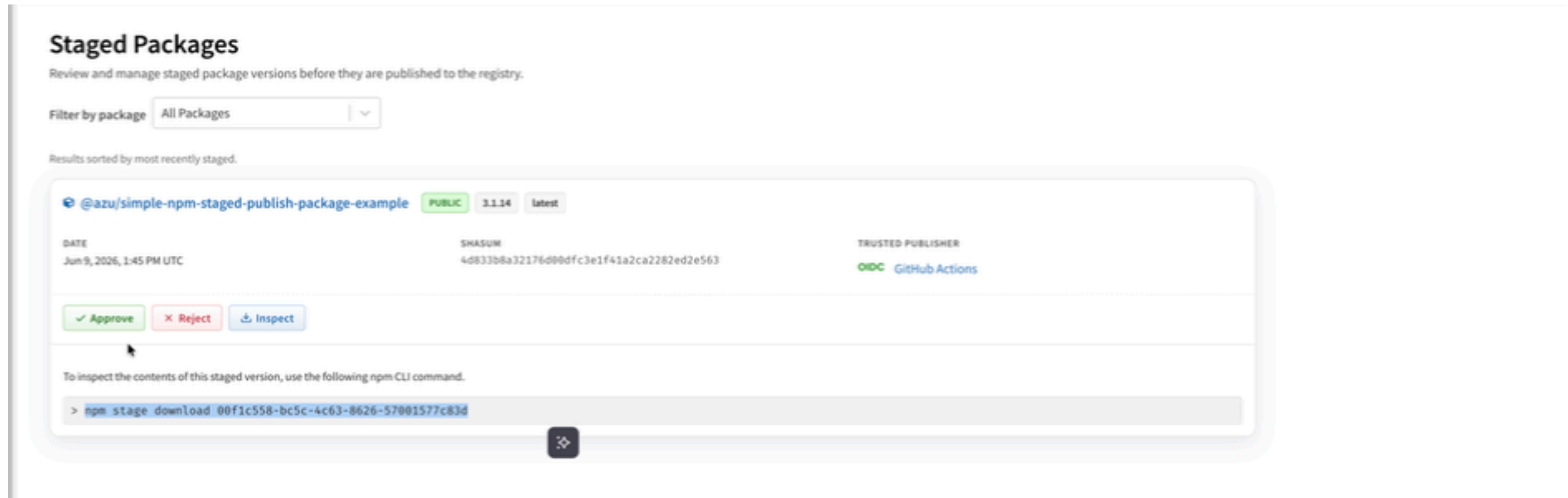
- npm publish は直接公開、npm stage publish は承認待ちにする
- OIDC設定で publish と stage publish のどちらを許可するか選べる
- stage publishのみ許可にすると、いきなりの公開が弾かれる
- npm 11.15.0以上 / Node 22.14.0以上、OIDC時のみ利用可

参考: docs.npmjs.com/staged-publishing

staged publishingのフロー

<code>npm stage publish</code>	# ステージング領域へ提出
<code>npm stage list</code>	# 一覧
<code>npm stage view <stage-id></code>	# 内容確認
<code>npm stage download <stage-id></code>	# 中身をDLして調査
<code>npm stage approve <stage-id></code>	# 承認（2FA必須）で公開

Staged Packagesの承認画面



- Trusted PublisherがOIDC / GitHub Actionsと表示される
- 中身を確認してから Approve で公開

staged publishingの課題

- 承認が1パッケージずつしかできない
- monorepo（40個一括）だと現実的でない
- CIから npm stage approve もできるが、それはnpmトークンが要る
- だからnpm / GitHubのどちらかのトークンは消し切るべき

まとめ

一連の流れを1個ずつ塞ぐ

1. ローカル: トークンを置かない / 強い権限を常駐させない
2. OIDC: トークンレス公開に強制する
3. Actions: PR + Environment + 承認ゲート
4. 権限昇格: ジャンプ地点に承認を挟む (SLSAの(F))
5. staged publishing: 公開直前にもう1枚レビューを足す

AIエージェント時代も同じ

- マスターが全権限を持つのではなく、権限を分散する
- 1タスク1エージェントにして、それぞれの範囲でオーソライズする
- AIに全部やらせれば、その分AIが狙われやすくなる

参考リンク

- npm OIDC: efcl.info/2025/09/07/npm-oidc/
- 公開例: github.com/azu/simple-oidc-example-package
- staged例: github.com/azu/simple-npm-staged-publish-package-example
- Bitwarden CLI侵害: socket.dev/blog/bitwarden-cli-compromised
- TanStack侵害(cache poisoning): tanstack.com/blog/npm-supply-chain-compromise-postmortem
- cache poisoning解説: adnanthekhan.com/posts/clinejection/
- SLSA Threats: slsa.dev/spec/v1.2/threats
- npm staged publishing: docs.npmjs.com/staged-publishing
- GitHub changelog: [pullrequesttarget](https://github.com/pullrequesttarget) and [environment](https://github.com/environment) [branch](https://github.com/branch) [protections](https://github.com/protect)

ありがとうございました

@azu_re